Transparent Orchestration of ReAct Agents (TORA)

Abstract

This study proposes **TORA (Transparent Orchestration of ReAct Agents)**, a new architecture designed to address a structural limitation of LLM-based agents built on the ReAct (Reason + Act) framework—namely, the inability to explain **why** a particular action was selected.

In existing ReAct-style agents, the LLM's generated *reasoning* is observable, but the underlying basis for choosing that reasoning, the alternative action candidates considered, and the decision-making process itself remain **opaque**. In multi-agent or multi-tool environments, this opacity compounds, making agent behavior difficult to predict, verify, or reproduce.

TORA tackles this issue not by interpreting the LLM's internal reasoning, but by **externalizing and transparently structuring the action-selection layer** outside the LLM.

The proposed method consists of four components:

1. Human-defined action scenarios written in natural language,
2. A transparent decision-making algorithm (TASL) that selects among these scenarios,
3. A ReAct agent executed as a black box according to the selected scenario, and
4. Logging of selection rationale, compared candidates, and execution results.

This design realizes a framework analogous to human decision-making, where
"action selection is transparent, while action execution remains a black box."

TORA is not intended to improve performance; rather, it provides a structural foundation for **accountability** in LLM agent action selection. As AI systems increasingly participate in societal decision-making processes, transparency in action selection is becoming a social requirement rather than a purely technical one. TORA offers a minimal and general architecture to meet this need.

---

1. Introduction

1.1 Background

LLM-based agents are capable of performing a wide range of tasks—including search, code generation, and external tool operation—with high proficiency. Among these, the ReAct (Reason + Act) framework has become widely adopted as a method that
solves complex tasks step-by-step by iteratively alternating between reasoning and tool execution.

A typical ReAct agent operates through the following cycle:

1. generating reasoning,
2. invoking tools when necessary, and
3. producing new reasoning based on the resulting observations.

However, this structure has an inherent limitation regarding **transparency in action selection**.
Although the reasoning text produced by the LLM is observable, the following remain opaque:

- why that particular reasoning was chosen,
- why a specific tool was selected,
- what alternative actions were considered.

In other words, **the core of the decision-making process remains a black box**.

Recent approaches such as AutoGPT, CrewAI, and LangGraph coordinate multiple agents, but this only amplifies the opacity:
each agent's hidden decision-making influences others, making the overall behavior increasingly difficult to predict, verify, or reproduce.

Similarly, protocols like MCP (Model Context Protocol) allow a single LLM to access many tools, but
the rationale behind tool selection remains buried inside the LLM's internal reasoning and cannot be externally understood.

In current ReAct-style agents, the only part that is explainable is
the intent of the initial prompt provided by the human,
while the agent's own action-selection process is fundamentally unexplainable.

Thus, across:

- single-agent systems,
- multi-agent systems, and
- multi-tool environments,

the question
"Why was this action selected?"
remains unanswered.

This lack of explainability significantly hinders
the understanding, reproduction, and improvement of agent behavior.

1.2 Problem Statement

The fact that an LLM's internal reasoning is a black box is not inherently problematic.
Humans also act without understanding the detailed workings of their neural systems.
However, humans can still explain:

"Why did I choose this action?"

Current LLM agents lack this structural capability.

In existing systems—including ReAct—:

- the reasoning process is output as natural language,
- but **the reason for choosing that reasoning** cannot be explained,
- and **the set of alternative actions considered** is not observable.

This limitation arises because the reasoning produced by an LLM is
not the actual reason,
but merely a natural-language sequence generated by the model.

The true basis for selecting that reasoning is encoded in hidden states and attention weights, making **the "reason for the reason" fundamentally inaccessible.**

Thus, even if an LLM writes:

> "I decided that Tool A should be used,"

this merely indicates
"the LLM wrote that sentence,"
not that it has provided a genuine explanation for its decision.

Furthermore, action execution (tool invocation) also depends on the LLM's internal reasoning, meaning that **both action selection and action execution are black-box processes.**

As a result, agent behavior in complex tasks becomes:

- **unpredictable**,
- **hard to verify**,
- **difficult to reproduce**,

severely undermining the explainability of action selection.

In multi-agent systems where multiple ReAct agents interact,
the opacity compounds:
each agent's hidden decision-making influences others, further deepening the lack of transparency.

At present, the only explainable component of an agent's behavior is
the human's initial prompt,
while the agent's own decision-making process remains unexplainable.

This leads to the central problem addressed in this study:

> **The issue is not interpreting the LLM's internal reasoning,**
> **but the absence of a mechanism that makes the action-selection layer itself transparent.**

1.3 Objective of This Study

This study adopts an architectural approach that does not attempt to interpret the LLM's internal reasoning. Instead, it aims to **externalize and make transparent the action-selection layer itself**.

The proposed architecture, **TORA (Transparent Orchestration of ReAct Agents)**, is structured as follows:

- action scenarios are defined by humans,
- ReAct agents are used as black boxes,
- action selection is delegated to **any transparent decision-making algorithm**,
- the rationale for selection is logged and made visible,
- and the selected scenario determines which ReAct agent is executed.

A key feature of TORA is that
it does not assume any specific decision-making algorithm.

The action-selection layer can be implemented using any transparent method—rule-based logic, scoring functions, utility maximization, or others.

The essence of TORA is to
guarantee transparency and explainability of action selection through an external layer,
achieving an architecture analogous to human decision-making:

> **“Action selection is transparent; action execution remains a black box.”**

This study does not aim to prevent AI misbehavior or address safety in general.
Its focus is strictly on
structuring transparency and explainability in action selection.

---

2. Related Work

2.1 ReAct Framework (Reason + Act)

- ReAct alternates between reasoning and acting.
- While the reasoning text is observable, **the rationale behind action selection is not**.
- The reasoning is not the “actual reason,” but merely text generated by the LLM.
- **No mechanism exists for making the action-selection layer transparent.**

Gap with TORA:
ReAct does not provide transparency in action selection.

2.2 Multi-Agent LLM Systems (AutoGPT / CrewAI / LangGraph)

- Multiple agents collaborate to solve tasks.
- However, each agent’s action-selection process remains a black box.
- Opacity compounds as agents influence one another.
- The lack of transparency becomes even more severe.

Gap with TORA:
No structure exists for making action selection transparent.

2.3 Multi-Tool LLMs (MCP / Toolformer)

- A single LLM is given access to many tools.
- However, the rationale for tool selection is buried inside internal reasoning.
- It cannot be understood from the outside.

Gap with TORA:
Tool-selection transparency is not addressed.

2.4 Explainable AI (XAI)

- Focuses on interpreting model internals (attention, attribution, probing).

- Attempts to explain the internal reasoning of LLMs.
- However, **the transparency of the action-selection layer is outside its scope.**

Gap with TORA:
XAI interprets internal reasoning, not action selection.

2.5 Cognitive Architectures (SOAR, ACT-R, etc.)

- Model human decision-making processes.
- However, their goals differ from making LLM agent action selection transparent.
- They are not aligned with the ReAct-style structure of modern LLM agents.

Gap with TORA:
They do not address transparency in action selection for modern LLM agents.

2.6 Summary: Identifying the Gap

Across existing research—whether focusing on:

- transparency of reasoning,
- interpretation of internal model states,
- multi-agent coordination, or
- optimization of tool use—

none address the core issue:

> **No existing work provides transparency for the action-selection layer itself.**

TORA fills this gap as
the first architectural proposal explicitly designed to make action selection transparent.

---

3. Proposed Method: Transparent Action Selection Layer (TASL)

3.1 Scope

The proposed TASL is an intentionally minimal architecture designed specifically to
make only the action-selection layer of LLM agents transparent.

Its scope is strictly limited to the following.

In Scope

- Action scenarios are **defined by humans in natural language**.
- ReAct agents are **designed by humans for each specific purpose**.
- Any transparent decision-making algorithm computes **only which scenario to select**.
- The rationale for action selection is **fully visible and logged**.
- The ReAct agent corresponding to the selected scenario is executed.
- Execution results are logged (but not used for learning).

Out of Scope

- Interpretation of the LLM's internal reasoning
- Transparency of the ReAct agent's internal reasoning
- Optimization of action execution
- Agent safety or misbehavior prevention (handled by PATI, not TASL)
- Agent learning or adaptation

By restricting its scope to **action-selection transparency alone**,
TASL maintains architectural purity and broad applicability.

3.2 Action Scenarios

In TASL, an action scenario is a **natural-language description of what the agent aims to accomplish**.

Action scenarios have the following properties:

- **One scenario corresponds to one clearly defined objective (What to do).**
- Scenarios must be **mutually comparable**.
- Each scenario is paired **one-to-one with a corresponding ReAct agent**.
- The internal implementation (How to do it) is irrelevant.
    - The ReAct agent's reasoning and tool usage remain a black box.
- TASL concerns itself only with **which scenario to choose**, not how it is executed.

Abstract Scenario Categories

To maintain generality, scenarios can be expressed using abstract categories such as:

- **Explore**
- **Acquire Information**
- **Assess Situation**
- **Generate Output**
- **Act on Environment**
- **Negotiate / Communicate**

Lightweight "sci-fi / game AI–style" examples for intuition

To make the structure more intuitive, here are concrete examples corresponding to the abstract categories:

- Explore
  → "Investigate an unknown area."
- Acquire Information
  → "Scan for nearby hostile units."
- Assess Situation
  → "Analyze current resource levels."
- Generate Output
  → "Summarize the next course of action."

- Act on Environment
  → “Open the door,” “Activate the device.”
- Negotiate / Communicate
  → “Talk to an NPC to extract information.”

These examples are **purely illustrative** and do not restrict TASL’s applicability.

3.3 Definition of ReAct Agents in TASL

This is a key conceptual shift in TORA/TASL.

A typical ReAct agent is a **general-purpose agent** that:

- performs arbitrary tasks,
- using arbitrary reasoning,
- with arbitrary tools,
- in an unconstrained manner.

In TASL, however, a ReAct agent is **redefined** as follows:

ReAct Agent in TASL

- A **dedicated agent corresponding to a specific action scenario**
- Its output scope (objective) is explicitly fixed
- Its internal reasoning and tool usage remain unrestricted (black box)
- However, **what it aims to achieve is fixed**
- This makes scenarios comparable as units of action selection

In other words:

> **The means may remain a black box, but the objective is fixed.**

This design ensures:

- ReAct’s flexibility (natural-language reasoning) is preserved
- The action-selection layer becomes transparent
- The common criticism “Why not just write code?” is addressed
  → Natural-language reasoning provides flexibility that code cannot replicate

---

3.4 Decision Module

The core of TASL is **transparent action selection**,
and it does not assume any specific algorithm.

Characteristics

- Any transparent decision-making algorithm can be used
- Rule-based logic, scoring, utility maximization, etc.
- Input: list of action scenarios

- Output: selected scenario + rationale (natural language or structured log)

Key Point

- **TASL does not prescribe the algorithm**
- **Transparency is the only requirement**

3.5 Action Execution (ReAct Execution)

The ReAct agent corresponding to the selected scenario is executed.

- The agent's internal reasoning remains a black box
- TASL does not intervene in execution
- Only the final result is returned
- Results are logged but not used for learning

3.6 Transparency Logging

The primary output of TASL is the **transparency log**.

The log includes:

- The selected scenario
- The rationale for selection
- The list of compared scenarios
- The executed ReAct agent
- A summary of the execution result

This ensures:

> **The action-selection process becomes fully understandable from the outside.**

3.7 End-to-End Flow

1. Input the current state
2. Enumerate action scenarios
3. Decision Module compares them
4. Log the rationale
5. Execute the corresponding ReAct agent
6. Return the result

---

4. Architecture (Final TORA/TASL Design)

4.1 Overall Structure

TASL (Transparent Action Selection Layer) is a minimal architecture designed to externalize the action-selection process of LLM agents and make it fully transparent.

TASL consists of the following four components:

1. **Action Scenario List**
2. **Scenario Selector (Transparent Decision Module)**
3. **ReAct Executor (Black-Box Executor)**
4. **Trace Logger (Transparency Log)**

Together, these components constitute the minimal structure required to realize TORA's core design principle:

> **"Action selection is transparent; execution remains a black box."**

4.2 Components

(1) Action Scenario List (Human-Defined Objectives)

- A set of **action scenarios defined by humans in natural language**
- Each scenario represents **one clearly defined objective (What)**
- Scenarios must be mutually comparable
- Each scenario is paired **one-to-one with a corresponding ReAct agent**

Examples (abstract):

- Explore
- Acquire Information
- Assess Situation
- Generate Output
- Act on Environment

Optional sci-fi/game-AI style examples for intuition:
"Investigate an unknown area," "Scan the surroundings," etc.

---

(2) Scenario Selector (Transparent Decision Module)

- **Input:** list of action scenarios + current state
- **Output:** selected scenario + rationale
- Any transparent decision-making algorithm may be used, including:
  - rule-based logic
  - scoring functions
  - utility maximization
  - other transparent methods
- **TASL does not prescribe the algorithm**
  → Any method is acceptable as long as transparency is ensured

This module is the **core of TASL**, responsible for externalizing the rationale behind action selection.

---

(3) ReAct Executor (Black-Box Executor)

- Executes the ReAct agent corresponding to the selected scenario

- The internal reasoning of the ReAct agent remains a black box
- The agent is free to use any reasoning or tools to achieve its objective
- TASL does not intervene in this process
- Only the final execution result is returned

In this architecture, the ReAct agent functions as an **actuator**.

---

(4) Trace Logger (Transparency Log)

The Trace Logger records:

- the rationale produced by the Scenario Selector
- the list of compared scenarios
- the executed ReAct agent
- a summary of the execution result

This ensures that:

> **The entire action-selection process becomes fully understandable from the outside.**

4.3 Data Flow (End-to-End Flow)

1. **State Input**
   The current state is provided to TASL.

2. **Scenario Enumeration**
   Human-defined action scenarios are listed.

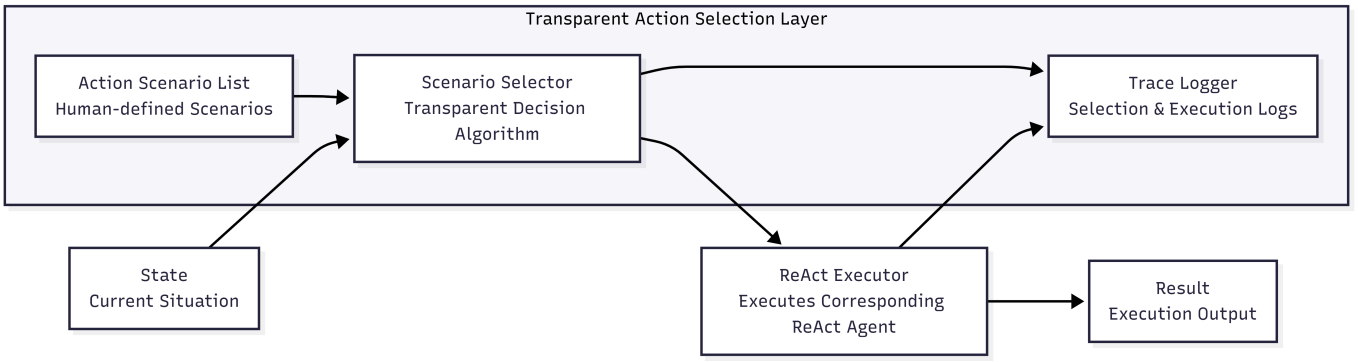
3. **Scenario Selection**
   The Scenario Selector determines which scenario to choose using a transparent algorithm and generates the rationale.

4. **Logging**
   The rationale and comparison results are recorded by the Trace Logger.

5. **Execution**
   The ReAct agent corresponding to the selected scenario is executed.

6. **Result Logging**
   The execution result is recorded by the Trace Logger.

7. **Output**
   The result is returned to the user (no learning occurs).



4.4 Why This Architecture Is Necessary

- The reasoning produced by ReAct is **not the actual reason**, but merely text generated by the LLM.
- The internal reasoning process of an LLM is **fundamentally unobservable**.
- Existing research does not address transparency in action selection.
- Only by externalizing the action-selection process can we realize the principle:
  “Action selection is transparent; action execution remains a black box.”

---

5. Evaluation

5.1 Objective

The goal of this study is not to interpret the internal reasoning of LLM agents, but to demonstrate that **TORA/TASL successfully externalizes and makes transparent the action-selection layer itself.**

Accordingly, this evaluation verifies whether the minimal implementation of TORA correctly performs:

- enumeration of action scenarios,
- transparent decision-making,
- logging of selection rationale, and
- execution of the ReAct agent corresponding to the selected scenario.

Since the internal reasoning of ReAct agents lies outside the scope of TORA,
a **black-box stub implementation** is used for the agents in this experiment.

5.2 Experimental Setup

Action Scenario List (Human-Defined)

Three action scenarios were defined in natural language:

- **escape**: move away from danger
- **scan**: scan for nearby threats
- **explore**: explore the surroundings

These satisfy TORA’s requirements:

- **one scenario = one objective**,

- scenarios are **mutually comparable**,
- each scenario is paired **one-to-one with a ReAct agent**.

Scenario Selector (Rule-Based)

To maximize transparency, a simple rule-based selector was used:

- If danger level is high (danger ≥ 7) → **escape**
- If an enemy is detected → **scan**
- Otherwise → **explore**

This satisfies TORA's requirement that
any transparent decision-making algorithm is acceptable,
making this the minimal viable configuration.

ReAct Executor (Black Box)

A dedicated ReAct agent was defined for each scenario:

- **EscapeAgent**
- **ScanAgent**
- **ExploreAgent**

Each agent is implemented as a stub, preserving TORA's principle that
action execution remains a black box.

5.3 Results

Three different states were provided as input,
and the full TORA end-to-end flow was executed.

Case 1: High Danger

```
State: danger=8, enemy_detected=True
Selected Scenario: escape
Reason: Danger level is high (8 >= 7)
ReAct Executor: EscapeAgent
```

→ The system correctly selected *escape* based on danger level,
and the rationale is explicitly stated in natural language.

Case 2: Enemy Detected

```
State: danger=3, enemy_detected=True
Selected Scenario: scan
Reason: Enemy is detected
ReAct Executor: ScanAgent
```

→ The system selected *scan* based on enemy detection,
and the rationale is transparently logged.

Case 3: No Danger, No Enemy

```
State: danger=2, enemy_detected=False
Selected Scenario: explore
Reason: No high danger and no enemy detected
ReAct Executor: ExploreAgent
```

→ The system correctly selected *explore* based on the situation.

5.4 Discussion

The results show that the minimal implementation of TORA/TASL satisfies the following:

(1) Transparent Action Selection

The system explicitly logs:

- which scenario was selected,
- why it was selected,
- which alternatives were compared.

(2) Black-Box Action Execution

- The internal reasoning of each ReAct agent remains hidden,
- but **which agent was executed** is fully transparent.

This matches TORA's design principle.

(3) One-to-One Mapping Between Scenarios and Agents

The mapping:

- escape → EscapeAgent
- scan → ScanAgent
- explore → ExploreAgent

functions exactly as intended.

(4) Flexibility of Natural-Language Scenarios

The descriptions of Action Scenarios are written in natural language,
confirming that **humans can freely define objectives**.

5.5 Summary

This experiment demonstrates that the minimal implementation of TORA/TASL successfully achieves:

- transparency of action selection,
- black-box execution of ReAct agents,
- one-to-one mapping between scenarios and agents,
- flexible objective definition via natural language.

Since TORA is not designed to improve performance but to
provide a structural guarantee of transparency in action selection,
the experiment fully satisfies its intended requirements.

Implementation details and code are provided in the accompanying repository.

---

6. Discussion

6.1 Balancing Transparency and Flexibility

The proposed TORA/TASL architecture simultaneously achieves
transparency in action selection and **flexibility in ReAct agent execution**.

- Action selection is made transparent through the Scenario Selector,
  which explicitly reveals which scenario was chosen and why.
- Meanwhile, the ReAct agent corresponding to the selected scenario
  is allowed to operate **freely as a black box**.

This structure enables TORA to balance two essential properties:

1. **Transparency (Explainability)**
    - The rationale for action selection is fully visible in an external layer.
    - Transparent rules or scoring functions can be refined and improved.
2. **Flexibility (Autonomy)**
    - ReAct agents may freely use reasoning and tools within the scope of their scenario.
    - Their internal reasoning remains a black box.

This mirrors human decision-making:

- We can consciously explain *what* we intend to do.
- But *how* our bodies execute the action is handled unconsciously.

TORA brings this structure into LLM agents.

6.2 Improving Transparency Over Time

A key feature of TASL is that
the decision-making algorithm can be freely replaced
as long as transparency is preserved.

While this study used a minimal rule-based selector,
many alternatives are possible:

- score-based selection
- utility maximization
- rule learning
- human-in-the-loop refinement

As long as transparency is maintained,
any algorithm fits within the TORA framework.

Moreover, because transparent logs are preserved:

- the validity of decisions can be inspected,
- inappropriate rules can be corrected,
- and the system can be aligned with human values.

This enables a **continuous improvement cycle**.

6.3 Benefits of Treating ReAct Agents as Black Boxes

The internal workings of ReAct agents may remain opaque—
this is fully aligned with TORA's design philosophy.

For example:

- EscapeAgent "searches for escape routes"
- ScanAgent "scans for nearby threats"
- ExploreAgent "explores unknown areas"

These processes correspond to unconscious motor or perceptual functions in humans.

TORA does **not** attempt to make these internal processes transparent.

The only part that must be transparent is:

> **"Why was this action selected?"**

The *execution* of the action is delegated entirely to the ReAct agent.

This separation ensures:

- transparency in action selection
- flexibility in action execution

without compromising either.

6.4 The Designer’s Trade-Off Boundary

TORA/TASL does not enforce a fixed level of transparency.

Instead, it is designed as a framework where:

> **Designers can freely choose the trade-off between transparency and automation.**

When prioritizing transparency:

- rule-based selection
- scoring functions
- human-defined value models (e.g., VBPM)

When prioritizing automation:

- LLM-generated scenarios
- LLM-based scoring
- LLM-driven action selection

Both approaches are valid.

TORA’s role is:

> **to structurally separate the layer that must be transparent,**
> **while leaving the degree of transparency to the designer.**

6.5 Positioning of TORA

TORA is not a performance-enhancing technique.
It is an architecture designed to **guarantee transparency in action selection**.

- It does not attempt to interpret ReAct’s internal reasoning.
- It externalizes the action-selection layer.
- It enables transparent decision-making.
- It preserves the flexibility of ReAct agents.

In this respect, TORA occupies a distinct position compared to existing XAI methods or multi-agent systems.

---

7. Conclusion

This study proposed **TORA (Transparent Orchestration of ReAct Agents)**, an architecture designed to externalize and make transparent the action-selection process of LLM agents.

TORA addresses a structural limitation of existing ReAct-based agents—
their inability to explain **why a particular action was selected**—by providing a
minimal and general framework in which:

- action scenarios are defined by humans in natural language,
- a transparent decision-making algorithm (TASL) selects among them,
- the corresponding ReAct agent is executed as a black box, and
- both the selection rationale and execution results are logged.

The goal of this work is not to interpret the internal reasoning of LLMs.
Instead, its novelty lies in adopting an architectural approach that
keeps internal reasoning as a black box while making only the action-selection layer transparent.

TORA is not a performance-enhancing technique;
it is a mechanism designed to provide **structural accountability**.

Humans cannot explain the neural-level details of how their bodies move,
yet they can explain:

> **"Why did I choose this action?"**

TORA brings this structure to LLM agents.

Although the broader demand for such an architecture remains uncertain,
the increasing involvement of AI systems in societal decision-making suggests that
explainability of action selection will become not merely a technical requirement
but a **social necessity**.

Clarifying:

- which action candidates were considered,
- why a particular action was chosen,
- and which agent executed it

constitutes the **minimum level of accountability** required for socially acceptable AI systems.

TORA provides a framework in which designers can freely choose the trade-off
between transparency and automation, offering a new foundation for
explainability in LLM agent action selection.